

Material de apoio ao módulo 3 - metrologia por imagem: processamento RGB com Python

Prof. Felipe Walter Dafico Pfrimer

1 Objetivo do módulo

Nos módulos anteriores, foram discutidos o acesso remoto a um sistema embarcado, a captura manual de imagens e a automação de rotinas experimentais. Neste módulo, a etapa seguinte do pipeline é analisada: o processamento das imagens. A execução desta atividade, entretanto, será feita no computador Windows do laboratório.

O objetivo principal é transformar uma sequência de imagens em uma tabela de dados e em um gráfico. Para isso, será usado um script Python já pronto, executado no computador Windows do laboratório, que calcula a média dos canais vermelho, verde e azul de cada imagem.

O que será feito

Etapa	Descrição
1	Abrir uma sequência de imagens de frutas.
2	Calcular a média dos canais R, G e B em cada imagem.
3	Salvar os resultados em um arquivo CSV.
4	Gerar um gráfico das componentes RGB ao longo da sequência.

2 Fundamentação teórica

Esta seção apresenta conceitos básicos para entender processamento de imagens pelo sistema de cores RGB e por que ele é útil para acompanhar mudanças visuais em imagens. O objetivo é fornecer um contexto para uma atividade prática, mostrando como as imagens digitais podem ser tratadas como dados experimentais.

2.1 Imagem como dado experimental

Uma imagem digital pode ser vista como uma matriz de pixels, como ilustrado na Figura 1. Cada pixel colorido possui três valores principais: vermelho, verde e azul. Esses valores são chamados de componentes RGB.

Componentes RGB

Canal	Interpretação
R	Intensidade de vermelho.
G	Intensidade de verde.
B	Intensidade de azul.

Em uma imagem com representação usual de 8 bits por canal, cada componente varia de 0 a 255. Valores próximos de 0 indicam baixa intensidade naquele canal; valores próximos de 255 indicam alta intensidade.

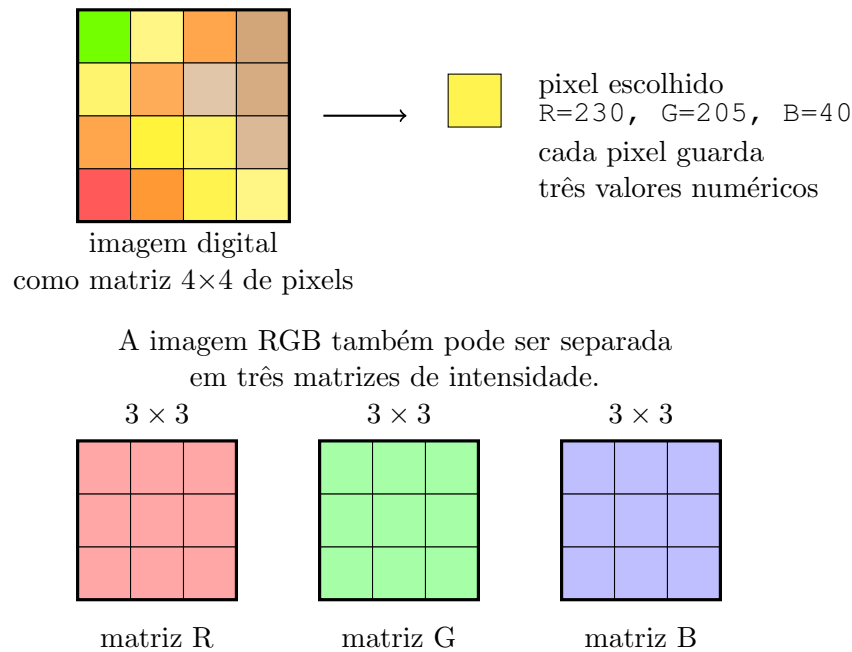


Figura 1: Representação simplificada de uma imagem digital como matriz de pixels e como três matrizes RGB.

Uma imagem RGB pode ser entendida como três matrizes sobrepostas: uma matriz para o canal vermelho, uma matriz para o canal verde e uma matriz para o canal azul. Essa representação permite acompanhar mudanças de cor por meio de medidas simples. Para cada imagem, podemos calcular a média de cada canal:

```
media_R = soma dos valores R / quantidade de pixels
media_G = soma dos valores G / quantidade de pixels
media_B = soma dos valores B / quantidade de pixels
```

Essas três médias resumem a imagem em três números. O resumo é simples, mas já permite construir curvas de variação ao longo de uma sequência de imagens.

Quando uma fruta muda de aparência, os valores médios desses canais também podem mudar. A análise RGB não substitui uma avaliação completa de qualidade, mas fornece uma primeira medida quantitativa simples e útil para discussão em metrologia por imagem.

O RGB não é o único sistema de cores possível. Dependendo do problema, outros espaços de cor podem ser mais adequados:

Outros sistemas de cor

Sistema	Ideia principal
Grayscale	Representa a imagem por uma única intensidade de cinza.
HSV/HSI	Separa matiz, saturação e intensidade, aproximando-se mais da forma como descrevemos cor visualmente.
CIELAB	Busca representar diferenças perceptuais de cor de forma mais uniforme.
YCbCr	Separa luminância de componentes de cor, comum em vídeo e compressão.

Neste módulo usaremos RGB porque ele é direto, fácil de visualizar e suficiente para uma primeira atividade. Em trabalhos mais avançados, comparar RGB com HSV, CIELAB ou

escala de cinza pode melhorar a interpretação.

2.2 Maturação, imagem e medição

Esta atividade foi inspirada no trabalho de Valesan (2022), que desenvolveu um dispositivo para controle de maturação de frutas e avaliação da eficiência de biofilmes comestíveis. A ideia central daquele estudo é muito próxima do pipeline discutido nesta disciplina: capturar imagens de forma periódica, armazenar os dados e aplicar processamento digital de imagem para acompanhar mudanças nos frutos.

2.3 Por que medir a maturação por imagem?

Após a colheita, frutas continuam sofrendo alterações físicas e químicas. Essas alterações podem aparecer como mudança de cor, brilho, textura, perda de massa e surgimento de manchas. Em muitas avaliações experimentais, a comparação entre frutas ainda é feita de forma visual e qualitativa, dependendo da percepção do avaliador.

Esse tipo de avaliação é importante, mas possui limitações:

- dois avaliadores podem interpretar uma mesma fruta de formas diferentes;
- a iluminação do ambiente pode alterar a percepção de cor;
- pequenas mudanças podem passar despercebidas;
- a comparação ao longo de muitos dias pode ser cansativa e irregular.

O processamento de imagem não elimina todos esses problemas, mas permite transformar parte da observação visual em números. Assim, uma mudança percebida como “a fruta escureceu” pode ser investigada por meio da variação dos canais de cor.

2.4 Processamento digital de imagem

Processamento digital de imagem, ou PDI, consiste em aplicar operações matemáticas sobre imagens digitais. Essas operações podem ter dois objetivos principais:

- melhorar uma imagem para interpretação humana;
- extrair informações numéricas para análise computacional.

Neste módulo, o objetivo de extrair informações numéricas é o mais importante. A imagem não é apenas uma figura para ser observada; ela é uma fonte de dados. Cada pixel possui valores numéricos, e esses valores podem ser organizados em tabelas e gráficos.

2.5 Região de interesse

Nem toda parte da imagem é útil para a medição. Em uma fotografia de uma fruta sobre fundo branco, por exemplo, muitos pixels pertencem ao fundo, não à fruta. Se esses pixels forem incluídos no cálculo, a média RGB pode representar mais o fundo do que o objeto de interesse.

Por isso, em PDI é comum definir uma região de interesse, também chamada de RI. A RI é a parte da imagem usada na análise. Em um experimento mais controlado, essa região pode ser escolhida por recortes fixos, máscaras, segmentação automática ou marcações feitas pelo usuário.

Na atividade deste módulo, usaremos uma versão simplificada dessa ideia: o script ignora pixels quase brancos. Dessa forma, a análise se aproxima mais da região ocupada pela fruta. A Figura 2 mostra, em uma imagem real do dataset, a diferença entre selecionar uma pequena região interna e tentar aproximar a fruta inteira.

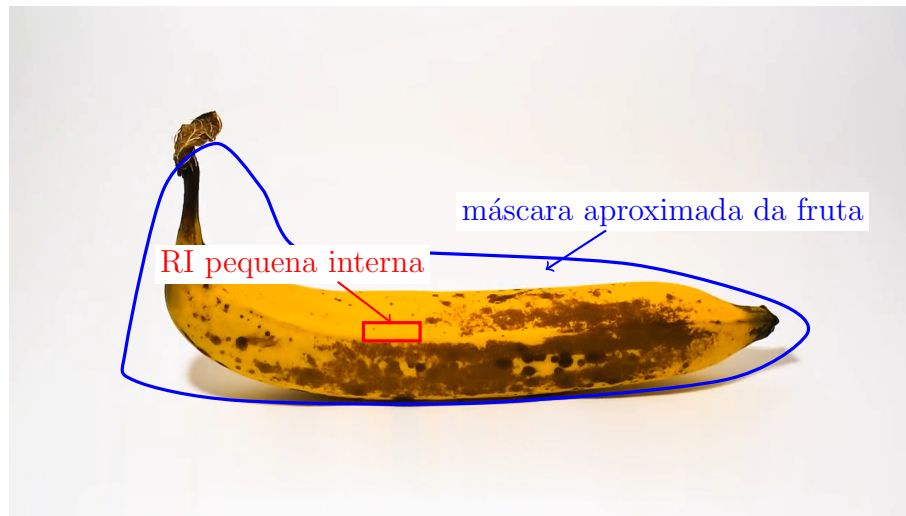


Figura 2: Exemplo real de duas formas de região de interesse em uma imagem do dataset: uma RI pequena no interior da banana e uma máscara aproximada da fruta inteira.

A RI pequena indicada na Figura 2 se aproxima da estratégia usada no trabalho de Valesan (2022), em que recortes internos eram escolhidos em regiões de cor aproximadamente uniforme. Essa abordagem reduz a influência do fundo e evita partes da fruta com sombra, brilho ou borda, mas mede apenas uma pequena região.

A máscara aproximada da fruta inteira, também mostrada na Figura 2, é mais parecida com a estratégia simplificada usada no script desta aula: em vez de escolher manualmente um quadrado, o programa tenta descartar pixels quase brancos e calcula a média dos demais pixels. Essa abordagem usa mais informação da imagem, mas pode incluir sombras, bordas, manchas e trechos de fundo que não foram removidos perfeitamente.

Comparação entre duas escolhas de RI na imagem `banana_016.png`

Região analisada	R	G	B	Comentário
RI pequena interna	253,88	181,29	0,04	Mede uma região amarela local, sem fundo aparente no recorte escolhido.
Máscara aproximada	179,62	126,26	35,47	Usa muito mais pixels da fruta; inclui manchas, bordas e sombras.

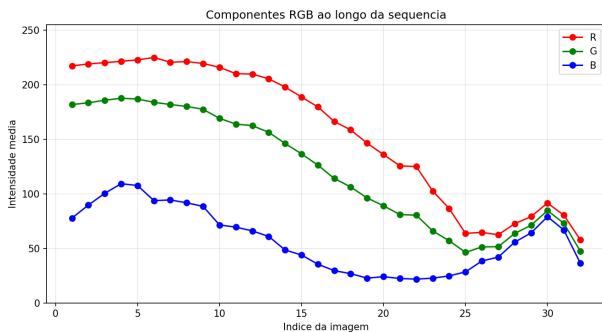
Os valores são diferentes porque as perguntas de medição também são diferentes. A RI pequena responde: “qual é a cor média desta região específica da banana?”. A máscara aproximada responde: “qual é a média dos pixels que o algoritmo considerou como não fundo?”. Em uma análise real, a escolha da RI deve ser definida antes do experimento e mantida constante para todas as imagens.

O script da atividade usa a máscara aproximada por padrão. Para explorar uma RI retangular fixa, existe a opção `--roi`. O comando abaixo usa uma RI de largura 80 pixels e altura 25 pixels, começando na posição `x=500` e `y=445` da imagem:

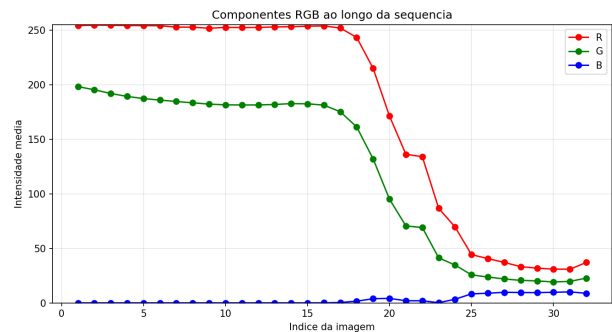
```
python 003_material_de_apoio/scripts/analise_rgb.py \
003_material_de_apoio/dataset_exemplo/banana \
--roi 500 445 80 25 \
--csv banana_roi.csv --grafico banana_roi.png
```

Essa RI fixa é útil para discutir o método, mas deve ser usada com cuidado: se a fruta mudar de posição, o retângulo pode deixar de apontar para a mesma região física da fruta.

A Figura 3 compara os gráficos produzidos pelas duas estratégias ao longo das 32 imagens de banana. A máscara aproximada da fruta inteira tende a produzir curvas mais suaves do que a RI pequena, pois usa muitos pixels da fruta. Mesmo assim, quando o fundo é removido de forma mais adequada, as curvas passam a variar mais do que variavam com o limiar antigo. A RI pequena é mais sensível ao ponto escolhido: ela pode destacar melhor uma região local da polpa, mas também pode variar bastante se a fruta mudar de posição, se a mancha passar pela região escolhida ou se o retângulo pegar sombra.



Máscara aproximada da fruta



RI pequena interna

Figura 3: Comparação entre o processamento usando uma máscara aproximada da fruta inteira e uma RI pequena interna.

3 Controle experimental e fontes de erro

Em um experimento real, não basta escolher um algoritmo. A forma de captura influencia diretamente os resultados. No estudo usado como referência, as imagens eram adquiridas periodicamente em uma câmara com ambiente controlado, junto com outras grandezas, como temperatura, umidade, luminosidade, dióxido de carbono e massa dos frutos.

Esse cuidado é importante porque a cor medida pela câmera pode mudar por motivos que não estão relacionados à maturação:

- variação da iluminação;
- sombras;
- mudança de posição da câmera;
- mudança de posição da fruta;
- ajuste automático da câmera;
- reflexos e brilho na superfície.

Por isso, técnicas como uso de fundo de referência, normalização, escolha de regiões de interesse e média móvel podem ser usadas para reduzir variações indesejadas. Nesta aula, aplicaremos apenas uma primeira aproximação: remover pixels de fundo quase branco e observar as médias RGB.

3.1 Relação com a atividade prática

O trabalho de referência acompanhava frutos ao longo do tempo real, com imagens capturadas periodicamente. No nosso caso, usaremos um subconjunto didático de imagens retiradas do FruitQ. A sequência foi organizada para representar uma progressão visual de qualidade, e não uma coleta temporal real.

Mesmo assim, o raciocínio experimental é o mesmo:

```
imagem -> região analisada -> valores RGB  
      -> tabela -> gráfico -> interpretação
```

A atividade é, portanto, uma versão reduzida do processo usado em estudos de maturação por imagem. O objetivo é entender a lógica da medição antes de avançar para técnicas mais complexas.

4 Dataset usado na atividade

As imagens desta atividade foram retiradas do FruitQ, um dataset público disponível no Kaggle para estudo de qualidade visual de frutas e hortaliças. O dataset completo é bem maior do que o material usado nesta aula e contém diferentes produtos, como banana, pepino, uva, caqui, mamão, pêssego, pera, pimentão, morango, tomate e melancia. As imagens são organizadas em classes visuais de qualidade, como boa, intermediária e deteriorada, dependendo da fruta.

Como o objetivo da aula não é treinar um classificador nem trabalhar com um banco de imagens grande, foi preparado um subconjunto didático pequeno. Isso reduz o tempo de processamento, facilita a visualização dos resultados e evita que a organização dos arquivos se torne o foco da atividade.

Neste material, foram usadas duas frutas:

- banana, usada no exemplo guiado;
- tomate, usado na atividade dos alunos.

Cada conjunto possui 32 imagens. O dataset completo permanece na pasta `FruQ-multi`, mas essa pasta não precisa ser usada pelos alunos. Para a aula, os arquivos foram copiados para `003_material_de_apoio` e renomeados com um padrão simples:

```
banana_001.png  
banana_002.png  
...  
banana_032.png
```

e

```
tomate_001.png
tomate_002.png
...
tomate_032.png
```

Esse tipo de nomenclatura é parecido com o que se espera em uma coleta real automatizada: todos os arquivos seguem o mesmo formato, mudando apenas o número da imagem.

4.1 Ordem das imagens

As imagens foram selecionadas respeitando a ordem numérica original do dataset e espaçadas de forma uniforme dentro de cada etapa. No conjunto didático, a sequência foi organizada assim:

Etapas da sequência

Índices	Etapa	Interpretação
1 a 11	boa	Fruta em melhor condição visual.
12 a 21	intermediária	Fruta com sinais intermediários de alteração.
22 a 32	deteriorada	Fruta com deterioração mais evidente.

A sequência não deve ser tratada como uma medição temporal real de laboratório. Ela representa uma ordem visual de alteração, útil para praticar processamento de imagem.

5 Por que ignorar o fundo branco?

Muitas imagens do dataset possuem fundo quase branco. Se a média RGB for calculada usando todos os pixels da imagem, o resultado pode representar mais o fundo do que a fruta. Isso é um problema experimental importante: nem todo pixel da imagem pertence ao objeto de interesse.

Para reduzir esse efeito, o script ignora pixels quase brancos. A regra usada é simples:

```
R >= 220, G >= 220 e B >= 220
```

Quando isso ocorre, o pixel é considerado fundo e não entra no cálculo da média.

Essa técnica é uma forma simples de limiarização, também chamada de *thresholding*. A limiarização é uma técnica clássica de segmentação em processamento de imagens: uma regra separa pixels em classes, por exemplo “fundo” e “objeto”, usando um ou mais valores de corte. Um método muito conhecido para escolher limiares automaticamente é o método de Otsu, que busca separar classes a partir do histograma de intensidades (OTSU, 1979). Livros de processamento de imagens, como Gonzalez e Woods (2018), também tratam a limiarização como uma técnica básica de segmentação.

A Figura 4 mostra o efeito da regra usada no script sobre a imagem `banana_016.png`. Os pixels pretos são aqueles que foram descartados por serem considerados fundo quase branco. Os demais pixels foram mantidos no cálculo.



Figura 4: Imagem de diagnóstico: pixels em preto foram desconsiderados pela regra $R \geq 220$, $G \geq 220$ e $B \geq 220$; os demais pixels foram mantidos no cálculo.

Na Figura 4, quase todo o fundo branco fica preto. Isso indica que a maior parte do fundo foi descartada antes do cálculo da média. Na imagem analisada, o limiar 220 descartou cerca de 83% dos pixels. Com o limiar anterior, 245, apenas cerca de 12% dos pixels eram removidos; muitos pixels de fundo ainda entravam na média. Esse fundo residual puxava os valores RGB para perto do branco e fazia as curvas variarem pouco, escondendo parte das mudanças visuais da banana.

Escolher o limiar é uma decisão experimental. Um limiar alto, como 245, só remove pixels muito brancos. Um limiar mais baixo, como 220, remove uma parte maior do fundo, mas pode começar a remover regiões claras do objeto, dependendo da cor da fruta e da iluminação. Por isso, o limiar deve ser escolhido observando imagens de teste, avaliando a imagem segmentada e mantendo a mesma regra para todas as imagens que serão comparadas.

O script permite testar outros valores com a opção `--limite-branco`. Por exemplo:

```
python 003_material_de_apoio/scripts/analise_rgb.py \
003_material_de_apoio/dataset_exemplo/banana \
--limite-branco 245 \
--csv banana_limiar245.csv --grafico banana_limiar245.png
```

Essa técnica não é perfeita. Ela pode falhar se o fundo não for branco, se a fruta possuir regiões muito claras ou se houver sombras fortes. Mesmo assim, é suficiente para mostrar uma ideia central: antes de medir, é necessário decidir qual região da imagem representa o objeto de interesse.

6 Execução do script

O aluno não precisa escrever o programa em Python do zero. O script já está pronto. A tarefa é executar o comando, conferir os arquivos gerados e interpretar o resultado.

O script está localizado em:


```
003_material_de_apoio/scripts/analise_rgb.py
```

As bibliotecas necessárias estão listadas em:

```
003_material_de_apoio/requirements.txt
```

No computador Windows do laboratório, se for necessário instalar as bibliotecas, abra o PowerShell na pasta do material e use:

```
python -m pip install -r 003_material_de_apoio/requirements.txt
```

Antes de executar o script, também é possível conferir se o Python está disponível:

```
python --version
```

6.1 Exemplo com banana

Para processar o conjunto de banana, execute:

```
python 003_material_de_apoio/scripts/analise_rgb.py \  
003_material_de_apoio/dataset_exemplo/banana
```

O comando gera dois arquivos:

```
resultado_rgb.csv  
grafico_rgb.png
```

O CSV é uma tabela com uma linha por imagem. O gráfico mostra como os canais R, G e B variam ao longo da sequência.

6.2 Atividade com tomate

Depois do exemplo, os alunos devem repetir o processamento com o conjunto de tomate:

```
python 003_material_de_apoio/scripts/analise_rgb.py \  
003_material_de_apoio/dataset_atividade/tomate \  
--csv tomate_rgb.csv --grafico tomate_rgb.png
```

O uso de `--csv` e `--grafico` permite escolher os nomes dos arquivos de saída.

7 Entendendo o script Python

O script `analise_rgb.py` foi organizado em funções pequenas. Isso facilita a leitura: cada função tem uma responsabilidade específica. O aluno não precisa memorizar todos os comandos, mas deve entender o caminho geral dos dados dentro do programa.

7.1 Bibliotecas usadas

No início do script aparecem as importações:

```
from argparse import ArgumentParser
from csv import DictWriter
from pathlib import Path
from PIL import Image
```

Cada biblioteca tem uma função:

Bibliotecas do script

Biblioteca	Uso no script
argparse	Ler opções digitadas no terminal, como <code>--csv</code> e <code>--grafico</code> .
csv	Salvar os resultados em uma tabela CSV.
pathlib	Trabalhar com caminhos de arquivos e pastas.
PIL	Abrir imagens e acessar os pixels.
matplotlib	Gerar o gráfico RGB.

7.2 Listagem das imagens

A função `listar_imagens` recebe uma pasta e retorna os arquivos de imagem encontrados nela:

```
def listar_imagens(pasta):
    extensoes = {".jpg", ".jpeg", ".png"}
    imagens = []
    for arquivo in Path(pasta).iterdir():
        if arquivo.suffix.lower() in extensoes:
            imagens.append(arquivo)
    return sorted(imagens)
```

O conjunto `extensoes` define quais formatos serão aceitos. O comando `iterdir()` percorre os arquivos da pasta. O `sorted` coloca os nomes em ordem. Como os arquivos foram padronizados como `banana_001.png`, `banana_002.png` e assim por diante, essa ordenação fica coerente com a sequência da atividade.

7.3 Identificação da etapa

Como os arquivos seguem uma sequência didática, a função `etapa_por_indice` associa cada intervalo de imagens a uma etapa:

```
def etapa_por_indice(indice):
    if indice <= 11:
        return "boa"
    if indice <= 21:
        return "intermediaria"
    return "deteriorada"
```

Assim, as imagens 1 a 11 são tratadas como boas, as imagens 12 a 21 como intermediárias, e as imagens 22 a 32 como deterioradas. Essa informação aparece depois na coluna etapa do CSV.

7.4 Abertura da imagem

A função mais importante é `media_rgb`. Ela começa abrindo a imagem:

```
imagem = Image.open(caminho).convert("RGB")
if roi:
    x, y, largura, altura = roi
    imagem = imagem.crop((x, y, x + largura, y + altura))
imagem.thumbnail((tamanho_maximo, tamanho_maximo))
```

O comando `Image.open` abre o arquivo. O `convert("RGB")` garante que a imagem será analisada com três canais de cor. Se o usuário informar `--roi`, o comando `crop` recorta uma região retangular antes do cálculo. O `thumbnail` reduz imagens muito grandes para acelerar o processamento, mantendo a proporção original.

7.5 Percorrendo os pixels

Depois, o script inicia acumuladores:

```
soma_r = 0
soma_g = 0
soma_b = 0
contador = 0
```

Essas variáveis guardam a soma dos valores R, G e B e a quantidade de pixels usados. Em seguida, o programa percorre os pixels:

```
for r, g, b in imagem.getdata():
    ...
```

Em cada repetição, `r`, `g` e `b` recebem os valores de um pixel.

7.6 Remoção simples do fundo

O script verifica se o pixel é quase branco:

```
fundo_branco = (
    r >= limite_branco and
    g >= limite_branco and
    b >= limite_branco
)
if ignorar_fundo and fundo_branco:
    continue
```

O valor padrão de `limite_branco` é 220. Se os três canais forem maiores ou iguais a esse valor, o pixel é considerado fundo. O comando `continue` pula esse pixel, ou seja, ele não entra na média.

Se o usuário executar o script com `--imagem-inteira`, a variável `ignorar_fundo` fica falsa e o programa usa todos os pixels, inclusive o fundo.

Se o usuário informar `--limite-branco`, o valor padrão 220 é substituído pelo valor escolhido. Isso permite testar limiares mais rígidos ou mais permissivos sem editar o código.

7.7 Cálculo das médias

Os pixels aceitos são somados:

```
soma_r += r
soma_g += g
soma_b += b
contador += 1
```

No final, a função retorna as médias:

```
media_r = soma_r / contador
media_g = soma_g / contador
media_b = soma_b / contador
return media_r, media_g, media_b, contador
```

Isso produz quatro informações: média de vermelho, média de verde, média de azul e quantidade de pixels usados.

7.8 Salvando o CSV

A função `salvar_csv` cria uma tabela com os resultados:

```
campos = ["indice", "arquivo", "etapa",
          "media_r", "media_g", "media_b", "pixels_usados"]
```

Cada imagem processada vira uma linha no CSV. Esse arquivo é importante porque permite analisar os dados depois em uma planilha, em outro programa Python ou em relatórios.

7.9 Gerando o gráfico

A função `salvar_grafico` separa os dados em listas:

```
x = [linha["indice"] for linha in linhas]
r = [linha["media_r"] for linha in linhas]
g = [linha["media_g"] for linha in linhas]
b = [linha["media_b"] for linha in linhas]
```

Depois, gera três curvas:

```
plt.plot(x, r, "r-o", label="R")
plt.plot(x, g, "g-o", label="G")
plt.plot(x, b, "b-o", label="B")
```

As letras `r`, `g` e `b` definem as cores das linhas no gráfico: vermelho, verde e azul.

7.10 Fluxo principal

A função `main` reúne todas as etapas:

1. lê os argumentos digitados no terminal;
2. encontra as imagens na pasta informada;
3. processa uma imagem por vez;
4. monta uma lista de resultados;
5. salva o CSV;
6. salva o gráfico;
7. mostra mensagens finais no terminal.

O trecho final:

```
if __name__ == "__main__":
    main()
```

significa que a função `main` será executada quando o arquivo for chamado diretamente pelo terminal.

7.11 Resumo do caminho dos dados

O fluxo completo do script pode ser resumido assim:

```
pasta de imagens
    -> lista de arquivos
    -> leitura de cada imagem
```

```
-> pixels RGB
-> remoção do fundo quase branco
-> média R, G e B
-> CSV e gráfico
```

8 Como interpretar a tabela

O arquivo CSV possui as seguintes colunas:

Colunas do CSV

Coluna	Significado
indice	Ordem da imagem na sequência.
arquivo	Nome do arquivo processado.
etapa	Etapa didática: boa, intermediária ou deteriorada.
media_r	Média do canal vermelho.
media_g	Média do canal verde.
media_b	Média do canal azul.
pixels_usados	Quantidade de pixels considerados após remover o fundo quase branco.

Para visualizar as primeiras linhas da tabela no terminal, use:

```
Get-Content resultado_rgb.csv -TotalCount 10
```

9 Como interpretar o gráfico

O gráfico apresenta três curvas: uma para o canal R, uma para o canal G e uma para o canal B. O eixo horizontal representa a ordem das imagens. O eixo vertical representa a intensidade média de cada canal.

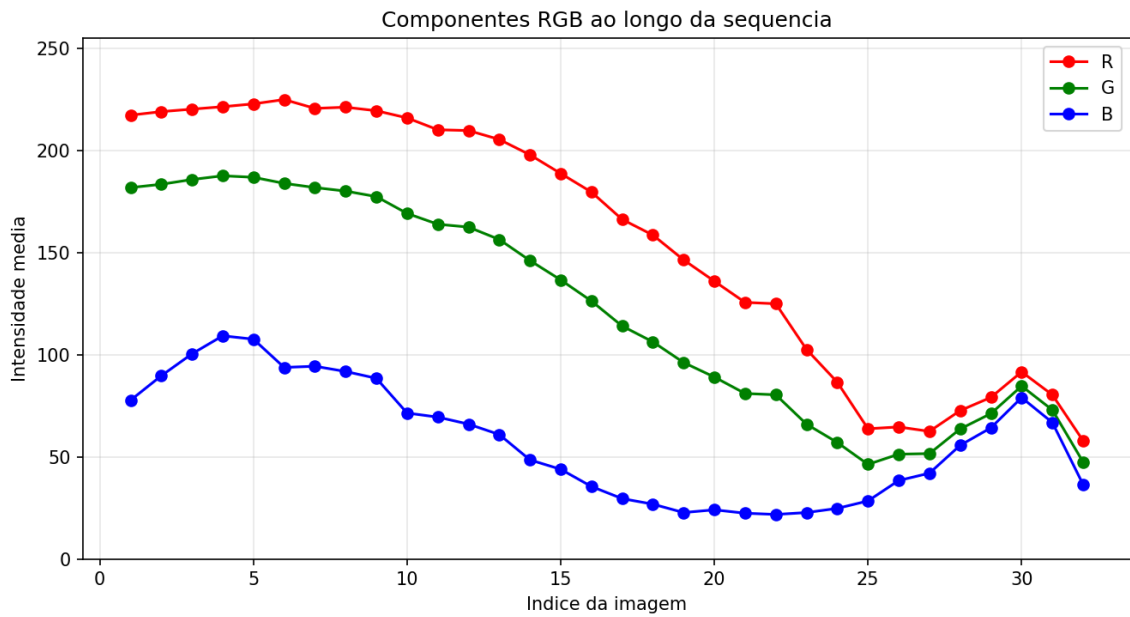


Figura 5: Exemplo de gráfico RGB gerado para a sequência de banana.

Durante a interpretação do gráfico, como no exemplo da Figura 5, observe:

- qual canal varia mais;
- se a mudança é gradual ou abrupta;
- se há diferença entre as etapas boa, intermediária e deteriorada;
- se algum ponto parece fora do padrão;
- se o fundo, a sombra ou a iluminação podem ter influenciado o resultado.

10 Limitações do método

O cálculo da média RGB é simples e fácil de entender, mas possui limitações. Ele resume a imagem inteira em apenas três números. Com isso, detalhes importantes podem ser perdidos.

Alguns fatores que podem afetar o resultado são:

- iluminação diferente entre imagens;
- sombras;
- mudança de posição da fruta;
- tamanho aparente do objeto;
- fundo da imagem;
- presença de mais de uma fruta na mesma imagem.

Essas limitações são parte importante da discussão. Em metrologia por imagem, medir não é apenas aplicar um algoritmo; é também controlar as condições de captura e entender as fontes de erro.

11 Relação com os módulos anteriores

Nos módulos 1 e 2, a captura de imagens foi discutida com comandos de terminal, scripts e automação em um sistema embarcado. Esses elementos fazem parte da etapa de aquisição dos dados.

Neste módulo, a análise RGB representa uma etapa posterior do pipeline:

```
experimento -> captura -> armazenamento -> processamento -> gráfico
```

Uma captura automatizada ajuda a produzir imagens mais comparáveis, porque reduz variações de horário, procedimento e intervenção manual. O processamento em Python transforma essas imagens em números que podem ser analisados e discutidos.

12 Questões para discussão

1. Qual canal RGB mudou mais ao longo da sequência?
2. A mudança entre as etapas parece gradual?
3. A média RGB separa bem as imagens boas, intermediárias e deterioradas?
4. O que mudaria se o fundo branco fosse incluído no cálculo?
5. Que melhoria poderia tornar a medição mais confiável?

13 Referência de apoio

GONZALEZ, Rafael C.; WOODS, Richard E. *Digital Image Processing*. 4. ed. New York: Pearson, 2018.

OTSU, Nobuyuki. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, v. 9, n. 1, p. 62–66, 1979.

SHOLZZ. *FruitQ Dataset*. Kaggle. Disponível em: <https://www.kaggle.com/datasets/sholzz/fruitq-dataset>.

VALESAN, Dyorgyo Pompermaier. *Dispositivo para controle de maturação de frutas e controle de eficiência de biofilmes comestíveis*. Trabalho de Conclusão de Curso (Bacharelado em Engenharia Eletrônica) – Universidade Tecnológica Federal do Paraná, Toledo, 2022.